TEST PLAN

Table of Contents

Introduction

This Test Plan defines the testing methodology and deliverables for Project 2: Object Detection for ECEN 4273It defines the objectives, scope, test schedule, risks, and the methods used to validate the system.

The deliverable system includes a trained YOLOv8 model, dataset-processing utilities, video detection, webcam detection, and GitHub-based CI workflows. The system must detect the following classes to within ≥ 75% accuracy as required by the assignment.

- Cat
- Dog
- Dalek
- Lightsaber
- Person

This document outlines test coverage for both functional and non-functional requirements.

1.1 Objectives

The YOLOv8 Object Detection System is a Python-based deep learning solution designed to identify objects in images and video streams and produce annotated outputs. The system incorporates YOLOv8 model training, dataset-processing utilities, video annotation tools, and live-stream detection using OpenCV. The test effort is responsible for validating the reliability and accuracy of the system to ensure it functions as intended across all supported input types. Because this is a student-driven project, the developer serves as both the creator and the tester of the product.

Phase 1 of the project will deliver a fully trained YOLOv8 model along with supporting scripts that allow a user to process datasets, perform training, run inference on videos, and execute real-time detection. This initial release must provide all required functionality listed in the assignment specification, including detection of cats, dogs, daleks, lightsabers, and people to within 75% accuracy. Meeting the functional requirements and achieving acceptable accuracy are considered more important than optimization or performance tuning in this phase.

1.2 Team Members

Resource Name	Role
Mark Hazen	Developer / Tester
Jimmy Nguyen	Developer/ Tester / Scrum Master

2 Scope

The test effort includes:

- All dataset-processing utilities, including scraping, cleaning, resizing, and filename normalization.
- YOLOv8 training pipeline, including configuration, dataset formatting, and training execution.
- Model inference on video files and real-time camera detection

- Accuracy evaluation on a dedicated test dataset with ≥ 75% accuracy for each required class.
- CI workflow functionality for automated checks.
- Documentation generation using pdoc.

3 Assumptions / Risks

3.1 Assumptions

1. The YOLOv8 training pipeline will run on a workstation with GPU support (NVIDIA RTX 4070 Ti or NVIDIA RTX 4080).
2. The dataset is available and contains enough images for each class to achieve target accuracy.
3. The final deliverable will be evaluated on a test dataset consistent with the project requirements.
4. GitHub will be used for version control and CI, and is assumed to remain stable.

3.2 Risks

The following risks have been identified and the appropriate action identified to mitigate their impact on the project. The impact (or severity) of the risk is based on how the project would be affected if the risk was triggered. The trigger is what milestone or event would cause the risk to become an issue to be dealt with.

#	Risk	Impact	Trigger	Mitigation Plan
1	Incorrect Labeling in Roboflow reduces model accuracy	High	Human error during annotation	Perform label review; re-label incorrect images before final training
2	Small dataset size results in poor detection	High	Limited number of labeled images	Add and label more samples
3	Model trains on CPU	High	CUDA not detected from missing NVIDIA drivers	Use torch.cuda.is_available() before training.
4	Dataset imbalance between classes	High	Uneven image collection	Collect additional samples

4 Test Approach

Testing will follow an iterative approach aligned with the assignment's sprint schedule, combining targeted test cases with exploratory testing to validate unknown behaviors.

Testing includes:

- **Unit Testing:** Validate individual dataset utilities and inference functions.
- **Integration Testing:** Verify that the full data to train to detection pipeline functions correctly.
- **System Testing:** Confirm real-time detection, video annotation, and overall reliability.
- **User Acceptance Testing:** Clone repository on a fresh system and confirm operational readiness.

4.1 Test Automation (MARK YOU NEED TO EDIT THIS AS NEEDED)

- Automated unit tests are executed via GitHub Actions as part of CI.
- When a change is made and pushed to the repository, the validation scripts are automatically ran to present the accuracy of the models for easy verification.
- Regression testing occurs through repeated CI runs after code changes.

5 Test Environment

The environment needed to perform all tests includes:

- **Hardware**
  - NVIDIA RTX 4070 Ti GPU / NVIDIA RTX 4080
  - Minimum 16 GB RAM
  - 5+ GB storage for datasets and model weights
- **Software**
  - Windows 10/11 or Linux
  - Python 3.10+
  - PyTorch 2.5.1+ with CUDA
  - Ultralytics YOLOv8
  - OpenCV
  - Git + GitHub CI

A webcam or Virtual Camera is required for live detection testing.

6 Milestones / Deliverables

6.1 Test Schedule

The initial test schedule follows……….

Task Name	Effort	Comments
Test Planning	1 Day	Creation of this Test Plan
Validate Requirements	1 Day	Review Assignment Document
Unit Test Dataset Scripts	1-2 Days	Image Scraper, cleaner, and resizer
Integration Test - YOLO Pipeline	1-7 Days	Validate that dataset from Roboflow can be trained using YOLOv8 without major errors and produces a valid weights file.
Functional Testing – Video Detection	1 Day	Annotated MP4 output
Functional Testing – Webcam Detection	1 Day	Real Time Detection and Annotation
System Testing / Regression	1 Day	After bug fixes

Final Documentation & Submission	1 Day	README, pdoc, test results

6.2 Deliverables

Deliverable	For	Date / Milestone
Test Plan Document	Instructor	12/5/2025
Trained Program	Instructor	12/5/2025
README Document	Instructor	12/5/2025
Auto-Generated Documentation	Instructor	12/5/2025
CI/CD Test Scripts	Instructor	12/5/2025
Training Cost Graph	Instructor	12/5/2025
Accuracy Report	Instructor	12/5/2025